

ГУАП

КАФЕДРА № 14

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Старший преподаватель	14.05.2026	Н.И. Синёв
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Ветвления в ассемблере

по курсу: Программирование на языках Ассемблера

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	2026	14.05.2026	А.Н. Корякин
1443		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Описание задачи

Написать программу с использованием ветвления в ассемблере Apple ARM 64.

Формализация

Если сумма целых чисел A, B и C меньше единицы, то наименьшее из этих трех чисел заменить суммой двух других. Результат программы должен выглядеть следующим образом, результат - значение переменных A, B и C после программы. Задача принимает три значения A, B, C - 64-битные целые знаковые числа, вводимые пользователем с клавиатуры. Ссылка на все исходники на GitHub: <https://github.com/Snctnm/Assembler>

Блок-схема

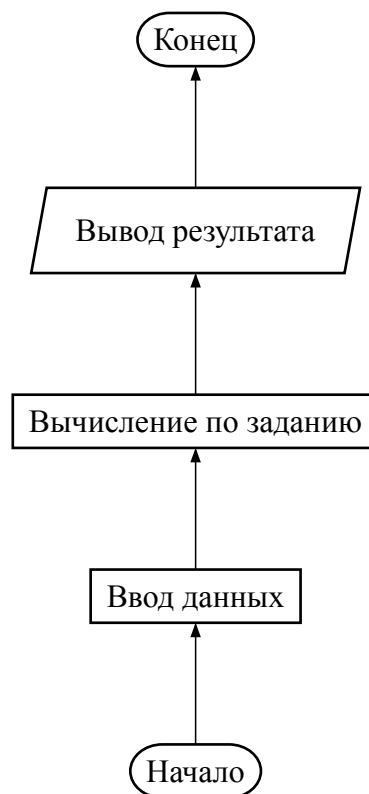


Рис. 1. Блок-схема программы

Исходный код программы

Код на ассемблере:

```
// Разветвления. Ассемблер.  
// Задание 9: Если сумма целых чисел A, B и C меньше единицы,  
// то наименьшее из этих трех чисел заменить суммой двух других.  
//  
// Формат ввода: три целых числа (знаковые 64-битные)  
// Формат вывода: измененные значения A, B, C  
  
.global _main  
  
.text  
.align 2
```

```

_main:
    // Выделяем место на стеке
    sub sp, sp, #96                // Выделяем 96 байт на стеке
    stp x29, x30, [sp, #80]        // Сохраняем fp и lr
    add x29, sp, #80                // Устанавливаем новый fp

    // Адреса переменных на стеке (отрицательные смещения от fp)
    sub x8, x29, #24                // Адрес для A (8 байт)
    sub x9, x29, #32                // Адрес для B (8 байт)
    sub x10, x29, #40               // Адрес для C (8 байт)

    // Передаём адреса переменных через стек для scanf
    mov x11, sp                    // Указатель на стек
    str x8, [x11]                  // Сохраняем адрес A на стеке
    str x9, [x11, #8]              // Сохраняем адрес B на стеке
    str x10, [x11, #16]            // Сохраняем адрес C на стеке

    // Ввод трех чисел
    adrp x0, input_str@PAGE
    add x0, x0, input_str@PAGEOFF
    bl _scanf

    // Загружаем значения из памяти
    ldr x8, [x29, #-24]             // x8 = A
    ldr x9, [x29, #-32]             // x9 = B
    ldr x10, [x29, #-40]            // x10 = C

    // Вычисляем сумму A + B + C
    add x11, x8, x9                 // x11 = A + B
    add x11, x11, x10               // x11 = A + B + C

    // Проверяем условие: если сумма < 1, то выполняем замену
    cmp x11, #1
    b.ge end_program                // Если сумма >= 1, пропускаем замену

    // Находим наименьшее число
    // Сравниваем A и B
    cmp x8, x9
    csel x12, x8, x9, lt             // x12 = min(A, B)
    csel x13, x9, x8, lt             // x13 = max(A, B) для информации

    // Сравниваем текущий минимум (x12) с C
    cmp x12, x10
    csel x12, x12, x10, lt           // x12 = min(prev_min, C)

    // Теперь x12 содержит наименьшее число
    // Нужно определить, какая переменная является наименьшей

    // Сравниваем A с наименьшим
    cmp x8, x12
    b.ne check_b                    // Если A не наименьший, проверяем B

```

```

// A - наименьшее, заменяем A на B + C
add x14, x9, x10           // x14 = B + C
str x14, [x29, #-24]       // Сохраняем новое значение A
b save_results

check_b:
// Сравниваем B с наименьшим
cmp x9, x12
b.ne check_c              // Если B не наименьший, значит C

// B - наименьшее, заменяем B на A + C
add x14, x8, x10           // x14 = A + C
str x14, [x29, #-32]       // Сохраняем новое значение B
b save_results

check_c:
// C - наименьшее, заменяем C на A + B
add x14, x8, x9            // x14 = A + B
str x14, [x29, #-40]       // Сохраняем новое значение C

save_results:
// Загружаем обновленные значения для вывода
ldr x8, [x29, #-24]        // x8 = A
ldr x9, [x29, #-32]        // x9 = B
ldr x10, [x29, #-40]       // x10 = C

end_program:
// Сохраняем значения на стеке для printf
mov x11, sp
str x8, [x11]              // Сохраняем A
str x9, [x11, #8]          // Сохраняем B
str x10, [x11, #16]        // Сохраняем C

// Вывод результата
adrp x0, output_str@PAGE
add x0, x0, output_str@PAGEOFF
mov x1, x8                 // Первый аргумент: A
mov x2, x9                 // Второй аргумент: B
mov x3, x10                // Третий аргумент: C
bl _printf

// Освобождаем стек и возвращаемся
ldp x29, x30, [sp, #80]    // Восстанавливаем fp и lr
add sp, sp, #96           // Освобождаем стек

// Завершение программы
mov x0, #0
mov x16, #1                // Системный вызов 1 (exit)
svc #0x80

// Секция данных

```

```

.section __TEXT,__cstring,cstring_literals
input_str:
    .asciz "%lld %lld %lld"

output_str:
    .asciz "Результат:\nA = %lld\nB = %lld\nC = %lld\n"

```

Тестирование

Выполним три сценария тестов по работе программы, с ручным вычислениям и работой программы.

Таблица 1. Расчёт суммы дальнейшей замены для трёх различных наборов

Набор тестовых данных	A	B	C	Сумма	Результат
1	4	2	2	$4 + 2 + 2 = 8 > 0$	Переменные не меняются
2	-1	-2	3	$-1 - 2 + 3 = 0 == 0$	$B \Rightarrow 3 - 1 = 2$
3	0	4	-4	$0 + 4 - 4 = 0 == 0$	$C \Rightarrow 0 + 4 = 4$

```

● snctm@MacBook-Pro-Aisen thirdLAB % ./lab3
4 2 2
Результат:
A = 4
B = 2
C = 2
● snctm@MacBook-Pro-Aisen thirdLAB % ./lab3
-1 -2 3
Результат:
A = -1
B = 2
C = 3
● snctm@MacBook-Pro-Aisen thirdLAB % ./lab3
0 4 -4
Результат:
A = 0
B = 4
C = 4
❖ snctm@MacBook-Pro-Aisen thirdLAB % 

```

Рис. 2. Результат тестов в программе

Выводы

В ходе выполнения лабораторной работы была успешно решена задача разработки программы на языке ассемблера ARM64 для суммирования трех вводимых переменных для дальнейшего сравнения с единицей, и если сумма меньше единицы, то после наименьшая переменная заменялась суммой двух оставшихся.

Что было сделано:

Формализация задачи - определены типы данных (знаковые 64-битные целые), этапы вычислений, для суммы, сравнения и замены. Разработка программы - написан код на ассемблере ARM64, включающий:

1. Выделение и освобождение стека с сохранением регистров fp/lr
2. Организацию ввода трёх чисел через scanf
3. Вычисление суммы
4. Сравнения с единицей
5. Нахождение минимального из трех переменных
6. Замены наименьшего суммой двух оставшихся
7. Вывод результатов через printf

Тестирование - проведено ручное тестирование для трёх наборов входных данных.

Результаты работы программы полностью совпали с теоретическими расчётами. Итог: Все поставленные цели лабораторной работы достигнуты. Программа корректно работает на платформе ARM64 macOS, правильно обрабатывает ввод знаковых чисел, выполняет необходимые арифметические операции с суммы, сравнения и замены, и выводит результат в требуемом формате.